

Use Cases – Sports Event Ticketing System

Sample Use Case					
Use case name:	User Registers and Logs into the System	ID:	1	Importance level:	High
Primary actor:	Fan (User)				
Short description:	This use case describes how a new user registers an account on the Sports Event Ticketing System and how an existing user logs in to access ticketing features.				
Trigger:	User visits the platform and chooses to register a new account or log in to an existing one.				
Type:	<u>External</u>				
Major Inputs			Major Outputs		
Description	Source		Description	Destination	
Full name	User		Registration confirmation message	User (Email / SMS)	
Email address	User		User account record	Database	
Phone number	User		Authentication token (session)	System / User	
Password	User		Login success / failure message	User	
National ID	User		Access to user dashboard	User	
Login credentials (email + password)	User		Error notification (invalid credentials)	User	
Major Steps Performed			→	Information for Steps	
1. User opens the web or mobile application.			→	Platform URL / App	
2. User selects "Register" to create a new account.			→	Register action	
3. User fills in: full name, email, phone, national ID, and password.			→	Full name Email Phone National ID Password	
4. System validates data format and checks for duplicate accounts.			→	User record (DB) Validation rules	
↳ If invalid data → System shows error and prompts user to correct it.			↔	Error message	
5. System saves new user record and sends confirmation email / SMS.			→	User account record Confirmation msg	
6. User selects "Login" and enters email and password.			→	Login credentials	
7. System authenticates credentials against the database.			→	User record (DB) Hashed password	
↳ If credentials incorrect → System displays login failure message.			↔	Failure message	
8. System generates auth token and grants access to the user dashboard.			→	Auth token Dashboard access	

Sample Use Case

Use case name:	User Searches and Browses Available Matches	ID:	2	Importance level:	High
Primary actor:	Fan (User)				
Short description:	This use case describes how a logged-in user searches for available sports matches using filters such as sport type, date, venue, or team, and views real-time seat availability.				
Trigger:	User logs in and navigates to the "Browse Matches" section to find a specific event.				
Type:	External				
Major Inputs		Major Outputs			
Description	Source	Description	Destination		
Sport type filter	User	Filtered list of matches	User		
Match date filter	User	Match details (date, time, teams, venue)	User		
Venue / stadium filter	User		User		
Team name filter	User	Interactive seat map	User		
Available matches list	Database	Available / unavailable seats	User		
Real-time seat availability data	Database / System	Ticket price per seat category	User		
		No results message (if none found)	User		
Major Steps Performed		→	Information for Steps		
1. User logs in and navigates to "Browse Matches".		→	Auth token	Browse Matches page	
2. System displays all upcoming available matches from the database.		→	Available matches list		
3. User applies filters: sport type, date, venue, and/or team name.		→	Sport type	Match date	Venue
			Team name		
4. System queries the database with applied filters and returns results.		→	Filtered matches list		
↳ If no matches found → System displays "No available matches" message.		↔	No results message		
5. User selects a specific match to view full details.		→	Match details		
6. System displays date, time, teams, venue, and ticket pricing.		→	Match details	Ticket price categories	
7. System loads interactive seat map with real-time availability. Green = available Red = sold / unavailable		→	Seat map data	Seat status (DB)	
8. User reviews seat map and pricing, then proceeds to booking if desired.		→	Selected match	Available seats	

Sample Use Case

Use case name:	User Books a Match Ticket	ID:	3	Importance level:	High
Primary actor:	Fan (User)				
Short description:	This use case describes how a logged-in user selects a seat for a specific match, proceeds through the payment process, and receives a booking confirmation. Ticket sales open 20 days before a match and close 3 hours before it starts.				
Trigger:	User selects an available seat from the seat map and clicks "Book Now".				
Type:	<u>External</u> <u>Temporal</u>				
Major Inputs			Major Outputs		
Description	Source	Description	Destination		
Selected match	User	Seat reservation (temporary hold)	Database / Seat map		
Selected seat(s)	User / Seat map				
Ticket quantity	User	Payment request	Payment gateway		
Payment method (Fawry / Paymob)	User	Payment confirmation	User / System		
		Booking confirmation record	Database		
Payment details	User	Booking confirmation notification	User (Email / App)		
Booking window status (open / closed)	System		User		
Major Steps Performed		→	Information for Steps		
1. User selects an available match and views the seat map.		→	Selected match Seat map		
2. System checks if the booking window is open (opens 20 days before, closes 3 hours before match).		→	Booking window status		
↳ If booking is closed → System displays "Booking unavailable" message.		↔	Closed booking msg		
3. User selects seat(s) and specifies ticket quantity.		→	Selected seat(s) Ticket quantity		
4. System temporarily reserves selected seats to prevent overselling.		→	Seat reservation (temp hold)		
5. System displays booking summary (match, seat(s), total price).		→	Booking summary Total price		
6. User selects payment method (Fawry or Paymob) and enters payment details.		→	Payment method Payment details		
7. System sends payment request to selected payment gateway.		→	Payment request Gateway response		
↳ If payment fails → System releases seat hold and notifies user.		↔	Payment failure msg Seat released		
8. System confirms payment, finalizes booking, and saves record in database.		→	Payment confirmation Booking record		
9. System sends booking confirmation to user via email and/or in-app wallet.		→	Confirmation notification Email / App wallet		

Sample Use Case

Use case name:	System Generates and Delivers QR Code Ticket	ID:	4	Importance level:	High
Primary actor:	System (Automated)				
Short description:	This use case describes how the system automatically generates a unique, encrypted, scannable QR code ticket after a successful booking payment, and delivers it to the user via email and the in-app wallet.				
Trigger:	Payment is confirmed successfully and a booking record is saved in the database.				
Type:	<u>External</u> <u>Temporal</u>				
Major Inputs			Major Outputs		
Description	Source	Description	Destination		
Confirmed booking record	Database	Unique encrypted QR code	System / User		
User information (name, email)	Database	QR code ticket stored in database	Database		
Match details (date, venue, seat number)	Database	Ticket delivered via email	User (Email)		
		Ticket added to in-app wallet	User (App wallet)		
Payment confirmation reference	Payment gateway	Delivery confirmation log	System log		
Unique ticket ID	System (generated)	Delivery failure alert (if fails)	System / Admin		
Encryption key	System				
Major Steps Performed		→	Information for Steps		
1. System detects successful payment confirmation from the payment gateway.		→	Payment confirmation ref		
2. System retrieves the confirmed booking record (user info, match details, seat number).		→	Booking record User name & email Match & seat details		
3. System generates a unique ticket ID for the booking.		→	Unique ticket ID		
4. System creates an encrypted QR code embedding ticket ID, match details, seat, and user identity.		→	Encrypted QR code Encryption key		
5. System stores the QR code ticket record in the database, linked to the booking.		→	QR ticket (DB record)		
6. System sends the QR code ticket to the user via email.		→	Email delivery User email		
↳ If email fails → System retries and logs failure; Admin is notified.		↔	Failure alert Retry log		
7. System adds the QR code ticket to the user's in-app wallet.		→	In-app wallet entry		
8. System logs successful delivery and updates booking status to "Ticket Issued".		→	Delivery confirmation log Status: Ticket Issued		

Sample Use Case

Use case name:	Admin Manages Events, Pricing, and Seat Allocation	ID:	5	Importance level:	High
Primary actor:	Admin (Ministry of Youth and Sports Staff)				
Short description:	This use case describes how an authorized admin logs into the admin dashboard to create, update, or remove sports events, manage seat allocation and ticket pricing, and generate reports on sales and attendance.				
Trigger:	Admin logs into the admin dashboard to add a new event, update an existing one, or review system reports.				
Type:	<u>External</u>				
Major Inputs			Major Outputs		
Description	Source	Description	Destination		
Admin credentials (username + password)	Admin	New / updated event record	Database		
Event details (name, date, teams, venue)	Admin / Ministry	Published event on platform	Users / Platform		
		Updated seat map and pricing	Database / Users		
Seat categories and capacity	Admin	Cancellation notifications (if event removed)	Users (Email / App)		
Ticket pricing per category	Admin		Admin		
Event status (active / cancelled)	Admin	Sales and attendance report	Admin		
Report filter criteria (date range, event)	Admin	Operation success / error message	Admin		
Major Steps Performed		→	Information for Steps		
1. Admin navigates to the admin dashboard and logs in with credentials.		→	Admin credentials		
2. System authenticates admin and grants access to the admin panel.		→	Auth token Admin panel access		
↳ If credentials invalid → System displays error message.		↔	Error message		
3a. Add New Event: Admin enters event name, sport type, date, time, teams, and venue. Sets seat categories, capacity, and ticket prices.		→	Event details Seat categories Ticket pricing		
↳ System saves the new event and publishes it on the platform.		→	New event record Published event		
3b. Update Existing Event: Admin selects an event and modifies required fields (date, pricing, seats).		→	Updated event record Updated seat map		
3c. Cancel Event: Admin marks event as cancelled.		→	Event status: Cancelled Cancellation notification		
↳ System sends cancellation notifications to all ticket holders and initiates refund process.		→	User notifications Refund trigger		
3d. View Reports: Admin selects report type and filters by date or event.		→	Report filter criteria		
↳ System generates and displays real-time sales, attendance, and revenue reports.		→	Sales report Attendance report		
4. System confirms the operation and displays success message to Admin.		→	Success / error message		